

ПРО КАНВАС «ПРОЦЕССЫ»

(наш n8n)

Полная техническая документация нодового движка
бизнес-процессов SuperApp6

*как устроен канвас · как двигается токен по нодам · что делает каждая нода · что делает каждая
ключевая функция · транзакции, локи, безопасность*

Версия документа: 2026-07-02 · Модуль: apps/api/src/modules/processes

Этот документ описывает движок «Процессы» так, чтобы ЛЮБОЙ разработчик, впервые открывший проект, понял: как устроен канвас, как двигается токен по нодам, что делает каждая нода, что делает каждая ключевая функция, где границы транзакций и локов, и как всё это безопасно работает в многоинстансном проде.

Все имена файлов, функций, полей и типов — реальные (сверены с кодом на дату выше). Пути даны относительно `apps/api/src/modules/processes`, если не сказано иное.

ОГЛАВЛЕНИЕ

0. Как читать этот документ
1. Что это и зачем (концепция «наш n8n»)
2. Словарь терминов (обязательно к прочтению)
3. Архитектура с высоты птичьего полёта (слои)
4. Карта файлов модуля
5. Модель данных (6 таблиц Prisma) — каждое поле
6. Формат документа-канваса (`nodes / edges / form / порты`)
7. Жизненный цикл определения: черновик → публикация → версии
8. Компилятор: документ → план исполнения (IR)
9. ДВИЖОК ИСПОЛНЕНИЯ — СЕРДЦЕ
 - 9.1 Что такое токен
 - 9.2 Логика движения токена (`adjacency`, порты, мульти-токен)
 - 9.3 Главный цикл: `kick` и `kickOnce` (3 фазы, лок, аренда, `io`)
 - 9.4 Завершение шага и продвижение (`completeStepAndAdvance`)
 - 9.5 Развилка (`fork`) и Слияние (`join`)
 - 9.6 Ветвь-локальный «Конец» и осушение (`drain` → `done`)
 - 9.7 Цикл «Перебрать список» (`loop.each`)
 - 9.8 Ожидающие шаги (задача / одобрение / пауза / очередь отдела)
 - 9.9 Хуки внешних событий (задача принята / удалена / отменена)
 - 9.10 `Claim` очереди / решение по одобрению / переназначение
 - 9.11 Крон-восстановление (таймеры, эскалация, застрявшие слияния)
 - 9.12 Локи и аренды — гарантии от гонок и двойного исполнения
 - 9.13 Отмена и ошибка инстанса
 - 9.14 Статусы и переходы
10. Поток данных (`variables`, `items[]`, `setVariables`, `{{...}}`)
11. Язык выражений (грамматика, операторы, функции)
12. Обработка ошибок (`onError`, повторы, порт «Ошибка»)
13. Триггеры (типы, реестр резолверов, условия, анти-runaway)
14. Безопасность (гейты ролей, `run-as-initiator`, SSRF, сейф кредитов)
15. СПРАВОЧНИК НОД — каждая нода с полями и примером
16. СПРАВОЧНИК ФУНКЦИЙ — по файлам, что/как/зачем
17. Сквозные примеры (рецепты)
18. Лимиты, стоп-краны, отложенное
19. FAQ и частые ошибки

0. КАК ЧИТАТЬ ЭТОТ ДОКУМЕНТ

Если ты хочешь БЫСТРО понять суть — читай главы 1, 3, 9.1–9.4. Это ядро. Если тебе править НОДУ — глава 15 (справочник нод) + глава 6 (формат). Если тебе править ДВИЖОК — главы 8, 9, 16 (справочник функций). Если ты пишешь НОВУЮ ноду — глава 6 + 15 + раздел «как добавить ноду» в 16.

Обозначения:

[ФАЙЛ] path/to/file.ts — где живёт код.

«Гвард» (status-guard) — условие в UPDATE ... WHERE, из-за которого параллельная попытка изменить ту же строку получает 0 обновлённых строк и «проигрывает» — так мы без блокировок БД гарантируем «ровно один раз».

1. ЧТО ЭТО И ЗАЧЕМ (КОНЦЕПЦИЯ «НАШ n8n»)

«Процессы» — это визуальный конструктор бизнес-процессов для организаций (B2B). Пользователь рисует на холсте схему из «нод» (шагов), соединяет их «проводами» (рёбрами), публикует — и процесс выполняется автоматически: создаёт задачи людям,

ждёт одобрений, ходит во внешние API (Kaspi, 1C, Telegram), зовёт ИИ, ветвится по условиям, идёт по нескольким веткам параллельно.

Это клон-в-духе трёх продуктов, взятых как эталоны:

- **n8n** — модель «триггер-нода + ноды-действия + провода», порты, cluster-агент.
- ComfyUI — «богатый документ в редакторе → компилируется в минимальный план».
- Camunda/Conductor — «token-движок»: процесс шагает токенами по графу, состояние живёт строками в БД, падение сервера ничего не теряет.

Почему написали CBOЁ, а не встроили n8n: n8n под лицензией Sustainable Use License (нельзя встраивать в свой продукт), Windmill — AGPL, Camunda 7 — EOL. Механику скопировали «чистой комнатой»: идеи да, код нет.

КЛЮЧЕВАЯ ИДЕЯ ДВИЖКА (запомни одну вещь):

Процесс — это НЕ последовательный скрипт. Это ГРАФ, по которому ползут ТОКЕНЫ.

Токен = «здесь сейчас находится управление». Один процесс может иметь НЕСКОЛЬКО токенов одновременно (параллельные ветки). Токен физически — это строка в таблице process_step_runs со status='active'. Движок берёт активные строки, выполняет их ноды, гасит строку и порождает новые строки на следующих нодах. Пока есть хоть одна активная строка — процесс идёт. Строк не осталось — процесс завершён.

2. СЛОВАРЬ ТЕРМИНОВ (ОБЯЗАТЕЛЬНО К ПРОЧТЕНИЮ)

Определение (Definition, ProcessDefinition)

Сам «процесс» как сущность: имя, видимость, статус (active/archived) и указатель на активную версию. Один процесс = одно определение = много версий.

Версия (Version, ProcessVersion)

Снимок документа + скомпилированный план. Публикация создаёт НОВУЮ версию.

Активна одна (currentVersionId). Модель Salesforce Flow: запущенные инстансы доживают на своей версии, даже если процесс перепубликовали.

Документ (Document, ProcessDocument)

Канвас как данные: плоский список нод (nodes), список рёбер (edges), анкета (form). Единственный источник правды; холст лишь рисует документ.

Нода (Node)

Один шаг/блок на холсте: «Задача человеку», «Если», «HTTP-запрос», «AI-Агент»...

Тип ноды берётся из реестра (ProcessNodeRegistry). У ноды есть config (настройки), входные/выходные порты.

Порт (Port)

Точка подключения провода. Выходные порты («main», «true»/«false», «approved»/«rejected», «success»/«error»...) — куда уходит токен. Тип порта: 'main' = поток токенов; ai_model/ai_memory/ai_tool/ai_output = подключение

под-ноды к AI-Агенту (не поток, а «оснастка»).

Ребро (Edge)

Провод: из выходного порта одной ноды (from, fromPort) во входной порт другой (to, toPort).

Триггер (Trigger)

Нода-точка входа. Фиксированного «Старт» НЕТ (модель n8n). Процесс начинается с триггер-ноды: «Запуск вручную» (start), «По расписанию», «Веб-хук», «Событие в SuperApp», «Telegram: входящее». Триггеров может быть несколько; каждый — своя точка входа.

План (Compiled Plan, IR)

Результат компиляции документа: минимальная исполняемая форма (nodes, adjacency, joinExpected, attachments, entryNodeIds). Именно план гоняет движок, НЕ документ.

Инстанс (Instance, ProcessInstance)

Один запуск процесса. Несёт анкету (variables), статус, счётчик шагов. Прикреплен к версии (versionId).

Шаг-ран (Step Run, ProcessStepRun)

Строка исполнения одной ноды в рамках инстанса. Активная строка (status='active') = ТОКЕН. Хранит тайминги («секундомер»), порт выхода (outcome), результат (output), ссылку на задачу (taskId) и т.п.

Токен (Token)

Абстракция «где сейчас управление». Физически = активный ProcessStepRun.

auto-нода

Движок проходит её сам, цепочкой, без ожидания извне («Если», «Уведомить», «HTTP», «Конец», триггеры). descriptor.auto = true.

Ожидающая нода (wait)

Возвращает {kind:'wait'} и «засыпает» строкой БД: ждёт внешнего сигнала — приёмки задачи, решения, таймера. descriptor.auto = false («Задача», «Одобрение», «Пауза», «Слияние»).

io-нода

Делает внешний ввод-вывод (HTTP/LLM/коннекторы). descriptor.io = true. Движок исполняет её ВНЕ инстанс-лока под «арендой шага» (см. 9.3) — долгий сетевой вызов не держит лок и не задвигается.

Инициатор (initiator, startedById)

От чьего имени бежит инстанс. Для ручного запуска — тот, кто нажал «Запустить». Для авто-триггеров — «runAs»-сотрудник, выбранный в триггере. ВСЕ действия (создание задач, уведомления, действия-ноды) идут от имени инициатора и его правами — процесс не может больше, чем его инициатор.

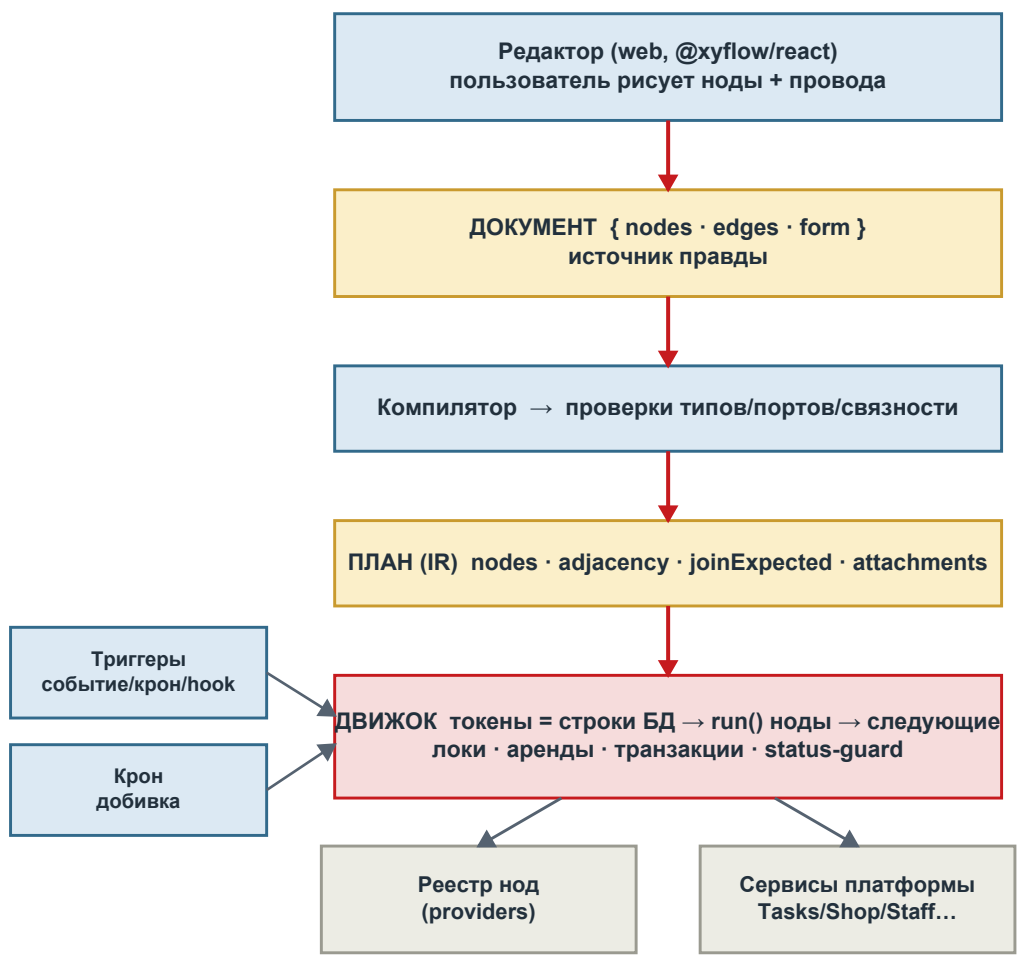
Инстанс-лок (kick-lock)

Redis-лок на конкретный инстанс. Сериализует bookkeeping (выбор/коммит шага), чтобы два kick одновременно не двигали один инстанс. Держится КОРОТКО, не поперх сети.

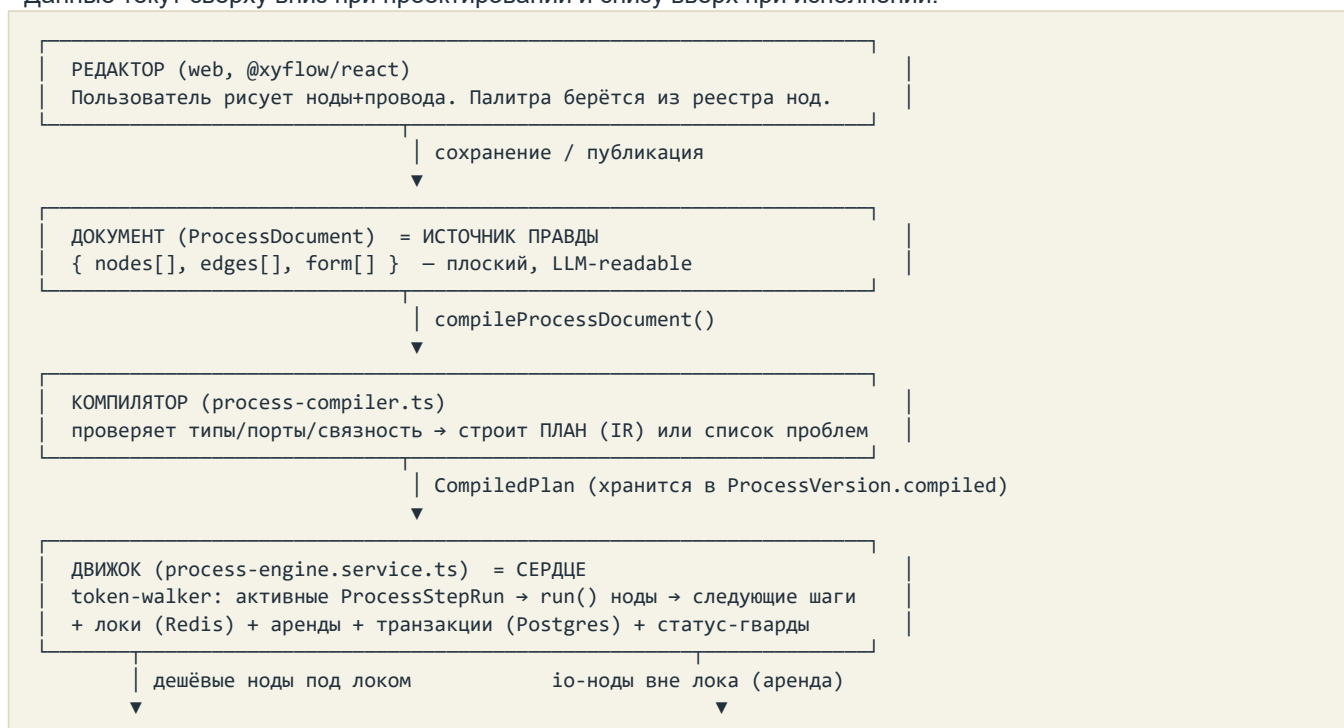
Аренда шага (lease, leasedUntil)

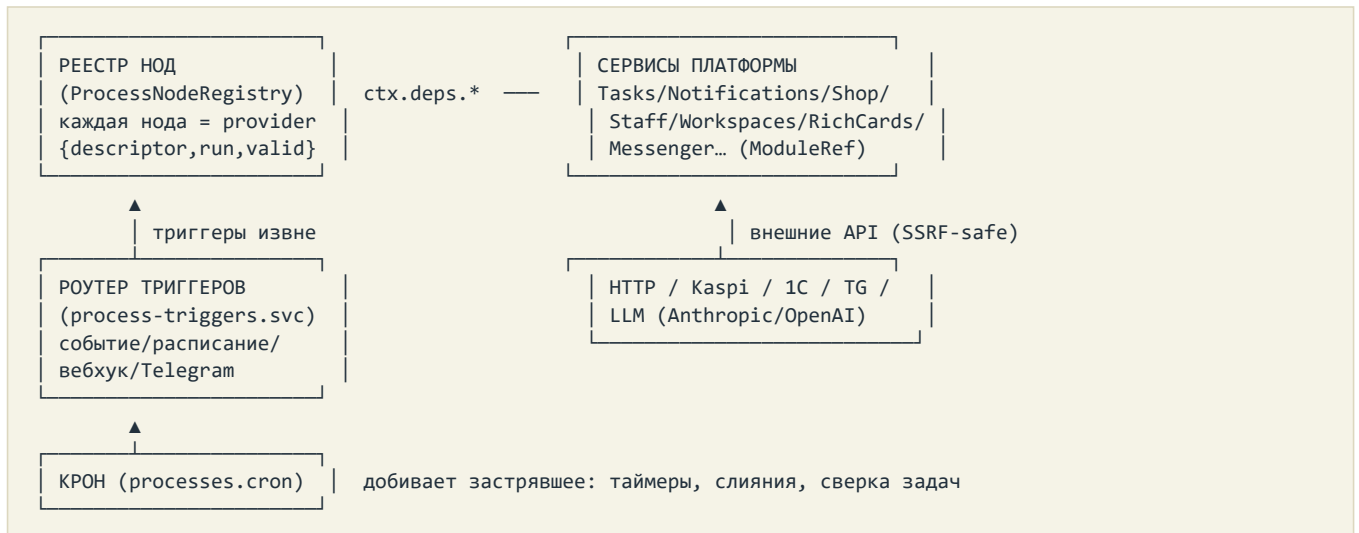
Метка времени на строке шага. Пока идёт внешний I/O, шаг «арендован» — другой kick/крон его не переисполнит, даже если инстанс-лок протух.

3. АРХИТЕКТУРА С ВЫСОТЫ ПТИЧЬЕГО ПОЛЁТА (СЛОИ)



Данные текут сверху вниз при проектировании и снизу вверх при исполнении:





Пять «китов», на которых стоит движок:

1. Документ — источник правды (не холст, не план).
2. План — неизменяемый снимок опубликованной версии (кэшируется, безопасно).
3. Токены — строки БД; состояние переживает рестарт.
4. Локи+аренды+гварды — «ровно один раз» без глобальных блокировок БД.
5. Крон — подстраховка: любой потерянный сигнал добывается по расписанию.

4. КАРТА ФАЙЛОВ МОДУЛЯ

apps/api/src/modules/processes/

processes.module.ts Регистрация модуля, DI, регистрация всех нод в реестре.

processes.service.ts CRUD определений, публикация, гейты ролей, запуск, инбокс, отчёты, сериализация DTO. «Фасад» модуля.

processes.controller.ts Тонкие HTTP-эндпоинты (Zod parse → сервис).

processes.cron.ts Крон-восстановление: дренаж застрявшего, таймеры, сверка.

process-engine.service.ts * ДВИЖОК: token-walker, локи, транзакции, продвижение.

process-compiler.ts Компилятор документ → план (IR) + валидация.

process-node.registry.ts Реестр типов нод (5-й платформенный реестр).

process-node.types.ts Все интерфейсы: descriptor, ctx, result, plan, cluster.

process-expression.ts Безопасный вычислитель выражений для {{ ... }}.

process-builtin-nodes.ts Встроенные ноды: триггеры, Задача, Одобрение, Если, Развилка/Слияние, Пауза, Перебрать, Задать данные, Уведомить, Конец + evalCondition().

process-service-nodes.ts HTTP-нода + вся SSRF-защита (safeFetch и т.д.) + креды.

process-action-nodes.ts Ноды-действия: карточка, сообщение, должность, роль, под-процесс (через ctx.deps.getService).

process-ai-nodes.ts AI: простой шаг + AI-Агент + под-ноды Модель/Память/Папсер + runAgentWithCluster().

process-ai-client.ts Чистый fetch к Anthropic/OpenAI (текст + tool-calling).

process-kz-nodes.ts Коннекторы KZ: Telegram, WhatsApp, SMS, Kaspi, 1C OData.

process-triggers.service.ts Роутер триггеров: событие/расписание/вебхук/Telegram.

process-crypto.ts AES-256-GCM шифр секретов для сейфа кредов.

packages/shared/src/

types/process.ts Типы документа/DTO/паспорта ноды (клиент+сервер).

constants/process.ts Лимиты, подписи статусов, списки событий/операторов.

apps/api/prisma/schema.prisma 6 моделей Process* (см. главу 5). apps/web/src/app/workspaces/[id]/processes/
Редактор канваса + списки + инстанс.

5. МОДЕЛЬ ДАННЫХ (6 ТАБЛИЦ Prisma) — КАЖДОЕ ПОЛЕ

[ФАЙЛ] apps/api/prisma/schema.prisma

5.1 ProcessDefinition — «процесс» как сущность

id uuid, PK.

workspaceId Организация-владелец (B2B: явное владение, не chokepoint).

name Имя процесса («Онбординг сотрудника»).

description Описание (необяз.).

visibility 'team' | 'admins'. 'admins' = «процессы для разработчиков»:

виден и запускается только admin+, скрыт от рядовых.

status 'active' | 'archived'. Архив блокируется бегущими инстансами (409).

currentVersionId Указатель на активную (опубликованную) версию. null = ещё не публиковали → запускать нечего.

createdById Кто создал определение. ВАЖНО: сбой процесса уведомляет и его (см. afterFailed, sflow#4) — для авто-запусков от служебного лица.

Индексы: (workspaceId, status).

5.2 ProcessVersion — снимок документа + план

id uuid, PK.

definitionId К какому определению относится.

version Порядковый номер (1, 2, 3...). @@unique([definitionId, version]).

status 'draft' | 'published' | 'superseded'.

document Json — сам канвас (nodes/edges/form). ИСТОЧНИК ПРАВДЫ.

compiled Json? — CompiledPlan. Заполняется ПРИ ПУБЛИКАЦИИ (у черновика null).

createdById Автор версии.

publishedAt Когда опубликована.

Инвариант: опубликованная версия неизменяема → её план можно кэшировать вечно (planCache в движке безопасен).

5.3 ProcessInstance — один запуск

id uuid, PK.

definitionId Процесс.

versionId * PIN: инстанс доживает на версии, на которой запущен, даже если процесс переопубликовали (Salesforce Flow).

workspaceId Организация.

status 'running' | 'done' | 'cancelled' | 'error'.

variables Json — анкета процесса + служебные ключи потока данных (_item, _loopIdx, _nodeId, _subprocessDepth, поля из «Задать данные»).

error Текст ошибки (если status='error').

stepsSpawned * Монотонный счётчик всех порождённых шагов. Инкрементится в транзакции спавна (O(1)) — стоп-кран от бесконечных циклов вместо COUNT(*) на каждом переходе (P8/A5). Стартовое значение 1 (входной шаг).

startedById Инициатор (для триггеров — runAs).

triggerType 'manual' | 'event' | 'schedule' | 'webhook' | 'telegram'.

wakeAt Будильник крона (зарезервировано под таймеры).

startedAt / finishedAt Тайминги («секундомер» всего процесса).

Индексы: (workspaceId,status), (definitionId,status), (startedById), (wakeAt).

5.4 ProcessStepRun — TOKEN / выполнение одной ноды

id uuid, PK.

instanceId К какому инстансу.

nodeId id ноды в документе версии.

nodeType Снимок типа ноды (списки не джойнят версию).

label Снимок подписи ноды (P7): журнал/детали не парсят документ.

status 'active' | 'done' | 'error' | 'cancelled'. active = TOKEN.

activated * side-effect шага уже отработал (задача создана / уведомление ушло / таймер поставлен). kick перезапускает только НЕактивированные ожидания; активированное ожидание «спит», его двигают хук/крон. Разделяет «нода ещё не отработала» от «ждёт человека/времени».

leasedUntil * Аренда для io-нод: пока идёт внешний вызов, шаг занят до этой метки.

startedAt / completedAt «Секундомер» шага (durationMs = разница).

outcome По какому порту ушёл токен ('main' / 'true' / 'approved' / 'error'...).

output Json — результат ноды (доступен следующим шагам как {{steps.<nodeId>}}).

error Текст ошибки шага.

taskId uuid? @unique — человеческий шаг → реальная задача Задачника.

assigneeId Исполнитель/согласующий шага.

sourceStepId Какой шаг породил этот (аудит токен-движка).

joinArrivals * Слияние: сколько токенов уже пришло (ждёт joinExpected).

departmentId Задача на отдел (очередь): члены отдела видят её в «Входящих» и claim'ят.

claimedById / claimedAt Кто и когда забрал задачу отдела.

decision Одобрение: 'approved' | 'rejected' (снимок решения).

deadlineAt SLA человеческого шага ИЛИ время побудки паузы.

escalatedAt Когда отправлена эскалация просрочки (дедуп — один раз).

Индексы: (instanceId,status), (assigneeId), (status,startedAt) [краны], (deadlineAt)

[таймеры/SLA], (departmentId) [очередь отдела].

5.5 ProcessTrigger — авто-триггеры (зеркалятся при публикации)

id uuid, PK.

definitionId / workspaceId

type 'event' | 'schedule' | 'webhook' | 'telegram'.

enabled Включён ли.

eventType * Денормализовано из config в колонку (P4): роутер событий фильтрует ИНДЕКСОМ, а не JSONB-path. Заполняется при публикации.

config Json. event:{nodeId,eventType,cond*}; schedule:{nodeId,everyValue,everyUnit}; webhook/telegram:{nodeId,credentialId?}.

webhookToken uuid? @unique — секрет публичного эндпоинта (стабилен между публикациями).

runAsUserId От чьего имени запускать процесс.

nextRunAt schedule: следующий запуск. lastRunAt: последний.

Индексы: (workspaceId,type,enabled), (type,enabled,nextRunAt) [schedule-крон],

(workspaceId,type,enabled,eventType) [роутер событий], (definitionId).

5.6 ProcessCredential — сейф секретов

id / workspaceId / name

type 'header' | 'basic' | 'bearer'.

data * Шифр AES-256-GCM: base64(iv).base64(tag).base64(ciphertext).

Секрет НАРУЖУ НЕ ОТДАЁТСЯ (DTO содержит только id/name/type).

Ключ шифрования выводится из JWT_SECRET (см. process-crypto.ts).

6. ФОРМАТ ДОКУМЕНТА-КАНВАСА (nodes / edges / form / порты)

[ФАЙЛ] packages/shared/src/types/process.ts

Документ (ProcessDocument) – плоский и LLM-readable by design:

```
{
  "nodes": [
    { "id": "trg", "type": "start", "label": "Запуск", "config": {},
      "position": { "x": 80, "y": 200 } },
    { "id": "task", "type": "human.task", "label": "Купить", "config": {
      "title": "Купить {{form.item}}", "assigneeMode": "initiator" } },
    { "id": "fin", "type": "end", "label": "Конец", "config": {} }
  ],
  "edges": [
    { "id": "e1", "from": "trg", "fromPort": "main", "to": "task", "toPort": "main" },
    { "id": "e2", "from": "task", "fromPort": "main", "to": "fin", "toPort": "main" }
  ],
  "form": [
    { "key": "item", "label": "Что купить", "type": "text", "required": true }
  ]
}
```

ProcessNode:

id Семантический стабильный id ('find_machine') — ключ рёбер и истории.

type Тип из реестра ('start' | 'human.task' | 'condition' | ...).

label Подпись (необяз.; если пусто — берётся title из паспорта).

config Именованные значения по configSchema типа (НЕ позиционные — урок ComfyUI).

position Только для холста; движок и валидация её игнорируют.

ProcessEdge:

from / fromPort Из какой ноды и её выходного порта (порт по умолчанию 'main').

to / toPort В какую ноду и её входной порт (по умолчанию 'main').

ProcessFormField (анкета, одна на инстанс):

key / label / type ('text' | 'number' | 'boolean' | 'date' | 'select') / required / options.

ПОРТЫ (ProcessPortType):

'main' Поток токенов. Провод main→main = «после этой ноды иди в ту».

'ai_model' Подключение под-ноды «Модель» к AI-Агенту.

'ai_memory' Подключение под-ноды «Память» к агенту.

'ai_tool' Подключение ноды-инструмента (или под-агента) к агенту.

'ai_output' Подключение под-ноды «Структурированный ответ» к агенту.

Правило компилятора: соединять можно только порты ОДНОГО типа (main→main, ai_model→ai_model и т.д.). main-порты образуют ГРАФ ПОТОКА; ai_*-порты образуют «оснастку» агента (attachments), НЕ участвуют в движении токена.

Паспорт типа ноды (ProcessNodeTypeDto) — то, что видит палитра и (позже) AI/МСР:

type/title/description/category/icon/tier + outputs[]/inputs?[] + флаги

terminal/trigger/multiOut/join/subNode + fields[] (виджеты формы). Это

«МСР-описание инструмента»: одна регистрация кормит палитру, валидацию и будущие AI-поверхности (Принцип 4 платформы).

7. ЖИЗНЕННЫЙ ЦИКЛ ОПРЕДЕЛЕНИЯ: ЧЕРНОВИК → ПУБЛИКАЦИЯ → ВЕРСИИ



активна одна версия (currentVersionId); запущенные инстансы доживают на своей (pin)

Модель — Salesforce Flow. Активна одна версия; правки идут в черновик v+1;

запущенные инстансы доживают на своей версии (pin по versionId).

```
создать процесс
|
v
[v1 draft] —сохранение канваса— [v1 draft] (мягкая валидация: issues, но
|                                   сохранять можно с проблемами)
|   publish (требуется issues=0 + исполнители — действующие члены)
v
[v1 published] — currentVersionId — ProcessDefinition
|   правка published-версии
v
[v1 published] + [v2 draft] (авто-открылся черновик следующей версии)
|   publish v2
v
[v1 superseded] + [v2 published] (инстансы на v1 доживают! pin)
```

Публикация (ProcessesService.publish, упрощённо):

1. Компилирует документ (compileProcessDocument). Если issues≠0 → 400.
2. Проверяет, что все выбранные исполнители/согласующие — действующие члены орг.
3. Помечает старую published как superseded, новую — published, обновляет currentVersionId.
4. syncTriggersOnPublish: зеркалит авто-триггер-ноды (schedule/webhook/event/telegram) в таблицу ProcessTrigger по ключу config.nodeId; webhook-токен стабилен между публикациями; для Telegram — best-effort setWebhook.
5. Компилятор пишет план в ProcessVersion.compiled.

8. КОМПИЛЯТОР: ДОКУМЕНТ → ПЛАН ИСПОЛНЕНИЯ (IR)

[ФАЙЛ] process-compiler.ts · функция compileProcessDocument(doc, registry)

Возвращает { plan: CompiledPlan | null, issues: [] }. plan=null, если есть хоть одна

проблема (публикация запрещена). Черновик может сохраняться и с issues — они просто подсвечиваются в редакторе.

Что делает по шагам:

- 1) УНИКАЛЬНОСТЬ. Дубли id нод и дубли ключей анкеты → issue. У select-поля без вариантов → issue.
- 2) ТИПЫ И КОНФИГИ НОД. Для каждой ноды:
 - неизвестный тип → issue;

- `configSchema.safeParse(config)` — Zod-проверка настроек; ошибки → issues с путём поля;
- `provider.validateConfig(config, doc)` — кросс-проверки против документа (напр. нода «Если» проверяет, что её поле реально есть в анкете);
- кладёт ноду в `plan.nodes` с полями: `type`, `label`, `config`, `terminal`, `auto`, `join`, `cluster` (есть ли вход `ai_model`), `onError/retryMaxTries/retryWaitMs` (`extractErrorHandling` из СЫРОГО `config` — реестр их стрипает из паспорта).

3) ТРИГГЕРЫ/КОНЕЦ. Нет ни одного триггера → issue. Нет «Конца» → issue.

`plan.entryNodeIds` = все триггер-ноды. `plan.startNodeId` = «Запуск вручную» (тип 'start'), иначе первый триггер (`back-compat`).

4) РЁБРА. Для каждого ребра проверяет: `from` существует, `to` существует, не петля

на себя, в триггер вести нельзя (у него нет входа), выходной порт существует, входной порт существует, ТИПЫ ПОРТОВ СОВПАДАЮТ.

- Если `iType='main'` → это ПОТОК: пишет в `plan.adjacency[from][port] = [ ...to ]`. Из одного порта — только ОДНО ребро, КРОМЕ ноды с `multiOut` (Развилка). `inDegree[to]++` (для `join`).
- Иначе (`ai_*`) → это ОЧАСТКА: пишет в `plan.attachments[to][iType] = [ ...from ]`. К одному `ai_`-входу — одна под-нода, кроме `multi`-порта (`ai_tool` у агента).

5) СВЯЗНОСТЬ ПОТОКА. BFS от ВСЕХ триггеров по `adjacency` → множество `reachable`.

У каждой достижимой ноды все её MAIN-выходы должны быть подключены (иначе токenu некуда идти) → issue «выход никуда не ведёт». `ai_*`-выходы под-нод — необязательны.

6) РАЗВИЛКА/СЛИЯНИЕ. У Развилки (`multiOut`) должно быть ≥ 2 ветки. У Слияния (`join`) —

≥ 2 входящих; `plan.joinExpected[nodId] = inDegree` (сколько токенов ждать).

7) КЛАСТЕР АГЕНТА. Под-нода должна быть подключена к агенту (иначе issue). Агент:

ровно одна Модель, не больше одной Памяти. Нода вне потока, не триггер и не под-нода → issue «недостижима».

8) А6 — ДЕТЕКТ ЦИКЛОВ АГЕНТОВ-ИНСТРУМЕНТОВ. Строит граф «агент → под-агенты (через

`ai_tool`)» и делает 3-цветный DFS. Цикл `A→B→A` → issue (рантайм-глубина ≤ 3 — это защита, но цикл конфигурации ловим на компиляции).

CompiledPlan (что реально гоняет движок):

startNodeId Точка ручного запуска (`back-compat`).

entryNodeIds Все триггеры (точки входа).

form Анкета.

nodes `nodId` → {`type`,`label`,`config`,`terminal`,`auto`,`join`,`cluster`,`onError`,`retryMaxTries`,`retryWaitMs`}.

adjacency `nodId` → `outputKey` → [следующие `nodId`]. Развилка → несколько.

joinExpected `nodId` слияния → сколько токенов ждать.

attachments `nodId` агента → тип порта → [id под-нод].

9. ДВИЖОК ИСПОЛНЕНИЯ — СЕРДЦЕ

[ФАЙЛ] `process-engine.service.ts` · класс `ProcessEngineService`

Стиль — как в кошельке (`WalletModule`): состояние в строках БД, переходы — атомарные `transaction`'ы со `status-guard`'ами. Модель — `Camunda/Conductor token-walker`.

9.1 ЧТО ТАКОЕ ТОКЕН

Токен = активная строка `ProcessStepRun (status='active')`. Она означает «управление находится на этой ноде». Инстанс жив, пока есть хоть одна активная строка. Когда активных строк не осталось — инстанс завершается (`drain` → `done`).

Жизнь строки-шага:

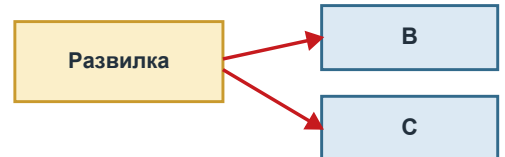
`create(status='active')` → [исполнение ноды] → либо `wait` (спит, `activated=true`),
либо `done` (`outcome=порт`) + порождение новых строк на следующих нодах.

9.2 ЛОГИКА ДВИЖЕНИЯ ТОКЕНА (adjacency, порты, мульти-токен)

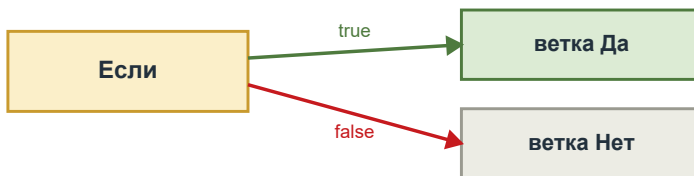
обычный переход (1 токен → 1 токен):



развилка (1 → N токенов):



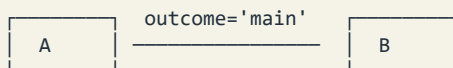
ветвление «Если» (токен по ОДНОМУ порту):



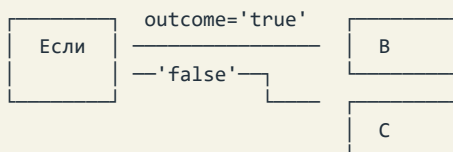
Нода вернула результат `{kind:'complete', outputKey}`. Движок берёт список следующих:

`nextIds = plan.adjacency[nodeId][outputKey]`

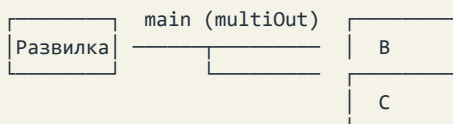
и на КАЖДЫЙ `nextId` порождает новую активную строку. Обычно `nextIds` = [одна нода]. У Развилки — несколько (`multiOut`). Если следующая нода — Слияние (`join`), строка не плодится, а «депонируется» в общий `join-шаг` (`joinArrivals++`).



обычный переход: 1 токен → 1 токен



ветвление: токен идёт по ОДНОМУ порту
(`adjacency['Если']['true']=[B]`,
`['false']=[C]`)

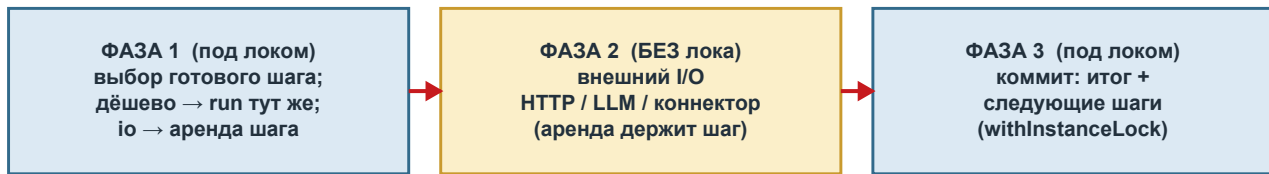


fork: 1 токен → N токенов
(по одному на каждое ребро)

Мульти-токенность: у инстанса одновременно может быть несколько активных строк. `kickOnce` обрабатывает ГОТОВЫЕ шаги по одному за проход, а `kick` крутит проходы, пока есть что делать — так за один толчок «разбираются» все готовые ветки.

9.3 ГЛАВНЫЙ ЦИКЛ: kick И kickOnce (3 ФАЗЫ, ЛОК, АРЕНДА, io)

kickOnce — один проход; kick крутит проходы до **maxAutoChain (100)**



инстанс-лок Redis (30с, короткий) держится ТОЛЬКО на фазах 1 и 3 — не поверх сети (P3)

kick(instanceId): крутит **kickOnce** до **maxAutoChain (=100)** раз или пока **kickOnce** не скажет «стоп». Искерпал бюджет → предупреждение в лог, доберёт крон (защита от бесконечной авто-цепочки в рамках одного толчка).

kickOnce(instanceId) – один проход. РАЗДЕЛЁН НА 3 ФАЗЫ, чтобы НЕ держать инстанс-лок

поверх сети (находка ревью P3):

ФАЗА 1 (ПОД инстанс-локом, `redis.withLock`, TTL 30с):

- читаем инстанс (не `running` → стоп);
- берём план (нет → `failInstance`);
- берём активные шаги (пусто → `completeInstanceIfDrained` → стоп);
- ищем ГОТОВЫЙ к исполнению шаг: `auto`-нода ИЛИ неактивированное ожидание, НЕ занятое арендой (`leasedUntil` в будущем → пропускаем – его крутит другой `kick`);
- нет готового → пробуем добыть застрявшее СЛИЯНИЕ (`tryForceFireStuckJoin`), иначе стоп;
- нашли готовый:
 - ДЕШЁВАЯ нода (не `io`) → `executeStepLocked: run()` + `commit` ПРЯМО ТУТ, под локом (`bookkeeping` без сети – быстро). Возврат «again» (крутим цикл).
 - `io`-нода (HTTP/LLM/коннектор/действие) → АРЕНДУЕМ шаг (`leasedUntil=now+200с`, `updateMany` со `status-guard` на свободную аренду). Отдаём наружу для Фазы 2.

ФАЗА 2 (БЕЗ лока): `runIoStep` → `provider.run()` – внешний вызов идёт без лока.

ФАЗА 3 (снова ПОД локом, `withInstanceLock` с РЕТРАЕМ захвата): `commitResult` – пишем итог/следующие шаги. Ретрай захвата – чтобы сигнал не потерялся (A7).

Почему так: держать 30-секундный Redis-лок поверх 3-минутного вызова к LLM —

катастрофа (лок протухает, второй `kick` исполняет ноду повторно, дорого/опасно).

Решение: дешёвые ноды под локом (быстро), а `io`-ноды — под АРЕНДОЙ шага вне лока.

Аренда (`leasedUntil`) живёт 200с и «держит» шаг, даже если инстанс-лок протух: другой `kick`/крон видит `leasedUntil>now` и пропускает. По истечении аренды крон переберёт.

`kickOnce` возвращает: `true` = «крутить дальше», `false` = «стоп» (пусто/лок занят/ инстанс не бежит).

Проигравший инстанс-лок просто уходит — активный `kick` доведёт.

9.4 ЗАВЕРШЕНИЕ ШАГА И ПРОДВИЖЕНИЕ (`completeStepAndAdvance`)

Это самая важная функция движения токена. Вся в ОДНОЙ транзакции, со `status-guard`ами.

Аргументы: `instanceId`, `plan`, `stepId`, `nodeId`, `outputKey`, `output?`, `extra?`, `setVariables?` Возврат: `AdvanceOutcome` = 'advanced' | 'consumed' | 'failed' | 'noop'.

Внутри транзакции по порядку:

1. `alive` = `updateMany(instance WHERE status='running' SET status='running')`. Это «первый замок» ВСЕХ писателей — `advance/cancel/fail` берут инстанс в одном порядке → нет ни осиротевшего `active`-шага в отменённом инстансе, ни дедлока. `count=0` → 'noop' (инстанс уже не бежит).
2. `claimed` = `updateMany(step WHERE id=stepId AND status='active' SET status='done')`,

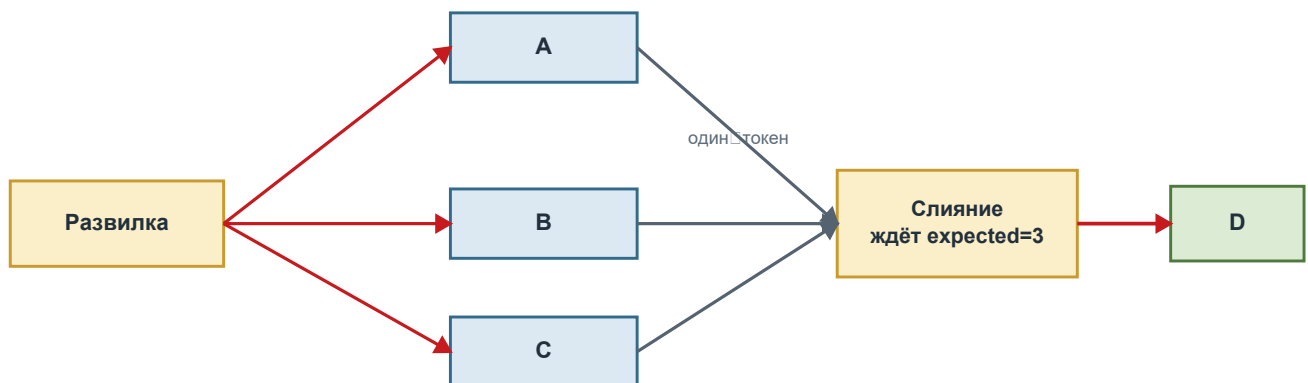
outcome, output, decision, leasedUntil=null, completedAt). count=0 → 'noop'
(шаг уже кто-то завершил — двойное продвижение проиграло на гварде).

3. setVariables (Ф5): если есть — read-merge-write в instance.variables В ТОЙ ЖЕ транзакции (нода «Задать данные» / индекс+элемент цикла). Параллельные ветки — last-write-wins.
4. Если нода terminal («Конец») → 'consumed'. Токен погашен (шаг уже done), но инстанс НЕ завершаем и соседние ветки НЕ трогаем (A2 branch-local end).
5. nextIds = adjacency[nodeId][outputKey]. Пусто или ведёт в несуществующую ноду → markErrorTx + 'failed'.
6. stepsSpawned += nextIds.length. Превысил maxStepsPerInstance (2000) → markErrorTx «возможен бесконечный цикл» + 'failed'. (Считаем фан-ауты токена, а не COUNT(*) — O(1) на переход.)
7. Спавним токены: на каждый nextId create(active). Если nextId — join-нода: не плодим, а инкрементим joinArrivals у существующего join-шага (создаём при ПЕРВОМ приходе) и будим его (activated=false). Все депозиты инстанса сериализованы инстанс-локом → гонки create-create нет; partial-unique индекс — фейл-сейф.
8. → 'advanced'.

После транзакции: если 'failed' → afterFailed (уведомления/события).

Мнемоника: «инстанс-гвард → шаг-гвард → (переменные) → терминал? → следующие → счётчик → спавн». Каждый гвард — это защита от гонки и двойного исполнения.

9.5 РАЗВИЛКА (fork) И СЛИЯНИЕ (join)



joinArrivals растёт 1→2→3; при arrivals≥expected — один токен идёт дальше

Развилка (parallel.split, multiOut): один входящий токен → на каждое исходящее ребро порождается свой токен. Далее ветки идут параллельно (движок разбирает их проходами kickOnce).

Слияние (parallel.join): ждёт, пока придут ВСЕ ветки. Реализация:

- Компилятор посчитал joinExpected = число входящих рёбер.
- Первый пришедший токен создаёт join-шаг (joinArrivals=1). Следующие — инкрементят joinArrivals (не плодят строку).
- join.run() смотрит ctx.join = {arrivals, expected}: arrivals≥expected → complete (идём дальше одним токеном); иначе wait (спим, следующий депозит разбудит).

A ┌
 B ┤ [Слияние] —(ждёт expected=3)— D joinArrivals растёт: 1 → 2 → ...
 C └ когда arrivals≥3 → один токен в D

ПРОБЛЕМА «зависшего слияния» (A1/A8) и её решение:

Если ветку увёл «Если»(false) или свой «Конец», join никогда не наберёт expected и инстанс повиснет БЕЗ сигнала. Решение — две подстраховки, обе на reachability (BFS canReach по adjacency):

- tryForceFireStuckJoin (СИНХРОННО, под локом в kickOnce, когда нет готовых шагов): если join уже созрел (пропущенный kick) ИЛИ ни один живой токен уже НЕ может дойти до него → добиваем с тем, что пришло (output.forced=true).
- sweepStuckJoins (КРОН, подстраховка для multi-instance/пропущенных): та же логика по всем активным join.

Так «вечное зависание» превращается в нормальное продолжение, а не в error.

9.6 ВЕТВЬ-ЛОКАЛЬНЫЙ «КОНЕЦ» И ОСУШЕНИЕ (drain → done)

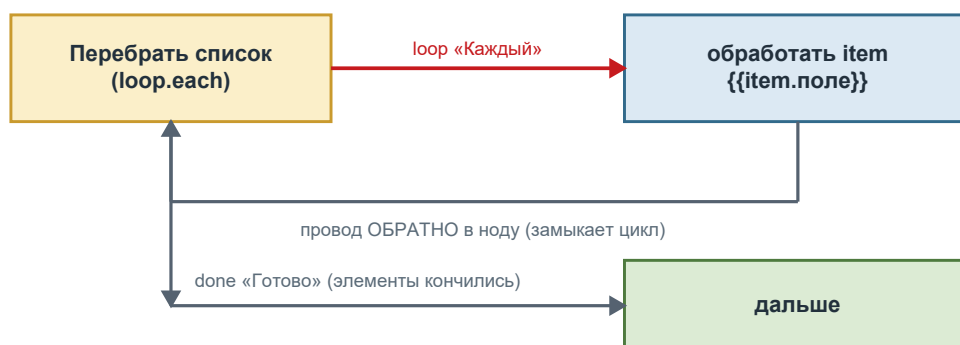
Продуктовое решение (выбор пользователя): нода «Конец» гасит ТОЛЬКО СВОЙ токен, а не весь процесс. Соседние параллельные ветки продолжают жить. Инстанс завершается (status='done'), лишь когда погаснут ВСЕ токены.

Механика:

- terminal-нода в completeStepAndAdvance → возвращает 'consumed' (шаг done, новых не порождает, но инстанс не трогаем).
- Когда kickOnce Фаза 1 видит «активных шагов нет» → completeInstanceIfDrained: updateMany(instance WHERE status='running' SET status='done') co status-guard от гонок → afterFinished.

```
└─ ветка 1 — [Конец] (consumed — гаснет свой токен)
Развилка
└─ ветка 2 — [задача] — [Конец] (consumed позже)
    ▲ пока эта ветка жива — инстанс running
    когда обе ветки погасли → активных строк 0 → drain → done
```

9.7 ЦИКЛ «ПЕРЕБРАТЬ СПИСОК» (loop.each)



Нода loop.each — последовательный ре-ентерабельный цикл (модель n8n Loop Over Items).

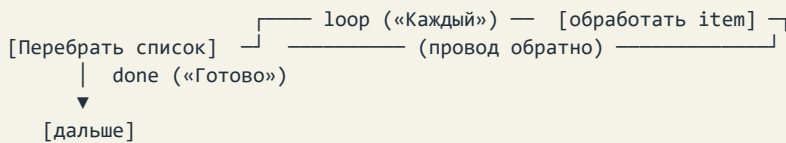
Она НЕ разворачивает список в параллельные токены (у нас есть human/wait-шаги, где это опасно) — она переисполняется по одному элементу.

Как это устроено:

- Состояние индекса — в variables._loopIdx_<nodeId>. Текущий элемент — в variables._item (доступен нодам как {{item.поле}}).
- run(): idx = variables[idxKey]; list = resolveValue(source); total = min(len, cap).
 - idx >= total → outputKey='done' (выходим), setVariables {idxKey:0, _item:null}.
 - иначе → outputKey='loop' (в ветку «Каждый»), setVariables

```
{idxKey: idx+1, _item: list[idx]}.
```

- Ветку «Каждый» ПОЛЬЗОВАТЕЛЬ ЗАМЫКАЕТ проводом ОБРАТНО в эту же ноду. Каждый заход выдаёт следующий элемент; когда кончились — уходим по «Готово».



Стоп-кран: каждая итерация — новые строки шагов, поэтому `maxStepsPerInstance` поднят

до 2000 (элементы × размер под-ветки). Ограничение осознанное: цикл ПОСЛЕДОВАТЕЛЬНЫЙ, не параллельный.

9.8 ОЖИДАЮЩИЕ ШАГИ (задача / одобрение / пауза / очередь отдела)

Ожидающая нода возвращает `{kind:'wait', patch, output}`. `commitWait` «усыпляет» шаг:

```
updateMany(step WHERE status='active' SET activated=true, leasedUntil=null,
            taskId?, assigneeId?, departmentId?, deadlineAt?, output).
```

`activated=true` → kick больше не трогает шаг. Его разбудит внешний сигнал (хук/крон).

Если инстанс отменили, пока создавалась задача (`patched.count=0` и был `taskId`) — гасим задачу-сироту (`updateTask status='cancelled'`).

Кто будит спящий шаг:

- Задача исполнителю → `onTaskCompleted` (хук Задачника) / крон-сверка.
- Задача на отдел (очередь) → `claimQueueStep` (сотрудник забрал).
- Одобрение → `decideApproval` (решение вынесено).
- Пауза / SLA → `runDueTimersAndEscalations` (крон-таймер).
- Слияние → депозит следующей ветки (`activated=false` → kick запускает `join.run`).

9.9 ХУКИ ВНЕШНИХ СОБЫТИЙ (задача принята / удалена / отменена)

`onTaskCompleted(taskId)`: задача-шаг полностью принята → шаг `done`, токен едет дальше.

Идемпотентно. Advance-фаза под инстанс-локом (A7): депозит в `join` сериализован против kick/крон. Порядок: найти активный шаг по `taskId` → взять инстанс-лок → перепроверить шаг под локом → `completeStepAndAdvance` → kick.

Двойная подстраховка: `TasksService` зовёт `onTaskCompleted` СИНХРОННО (`ModuleRef`-токен `'ProcessesService'`) И эмитит событие в `EventBus`; плюс крон-сверка wait-шагов с их задачами — на случай потерянного сигнала (`at-most-once` шина).

`onTaskDeleted(taskId)`: связанную задачу удалили → `failInstance` («Связанная задача удалена») — процесс не может продолжаться.

`onTaskCancelled(taskId)`: задачу-шаг отменили → `failInstance` («Задача шага отменена»).

9.10 CLAIM ОЧЕРЕДИ / РЕШЕНИЕ ПО ОДОБРЕНИЮ / ПЕРЕНАЗНАЧЕНИЕ

`claimQueueStep(userId, instanceId, stepId)`: забрать задачу отдела из очереди.

Проверяет, что шаг ждёт claim (`taskId=null, departmentId≠null`), что `userId` — член отдела (`relationTuple department#member`). Создаёт реальную задачу `claimer'y` (Постановщик — инициатор). Захват: `updateMany(step WHERE taskId=null SET taskId=...)` — первый выигрывает, остальные → `count=0` → гасят свою лишнюю задачу. Модель `Camunda candidate-group`.

`decideApproval(userId, instanceId, stepId, 'approved'|'rejected')`: решение по одобрению. Проверяет, что решает назначенный согласующий (`assigneeId=userId`).

Advance под инстанс-локом; перепроверяет шаг под локом → `completeStepAndAdvance` с `outputKey=decision` (токен идёт по ветке `approved/rejected`) → kick.

reassignStep(instanceId, stepId, newUserId): переназначить исполнителя активной задачи-шага (manager+). Проверяет, что новый — действующий член. Зовёт `tasks.reassignExecutorTrusted` (снимает старого, ставит нового, награда переморожена), обновляет `assigneeId` шага.

9.11 КРОН-ВОССТАНОВЛЕНИЕ (таймеры, эскалация, застрявшие слияния)

[ФАЙЛ] `processes.cron.ts` (под Redis-локом, один инстанс кластера) + движок.

runDueTimersAndEscalations():

- 1) Паузы, чьё время вышло (`nodeType='delay', deadlineAt≤now`) → advance под инстанс-локом → kick. Скан по индексу `deadlineAt`.
- 2) Просроченные человеческие шаги (`human.task/approval, deadlineAt≤now, escalatedAt=null`) → эскалация инициатору+исполнителю (дедуп через `escalatedAt status-guard`).

sweepStuckJoins(): добить зависшие слияния (см. 9.5).

Крон-восстановление также ДРЕНАЖИТ застрявшие/waiting-шаги БАТЧАМИ (курсор, BATCH=500) и сверяет wait-шаги с их задачами (`done/cancelled/deleted`) — идемпотентные подстраховки от потерянного сигнала.

9.12 ЛОКИ И АРЕНДЫ — ГАРАНТИИ ОТ ГОНОК И ДВОЙНОГО ИСПОЛНЕНИЯ

Три уровня защиты, работают вместе:

1. ИНСТАНС-ЛОК (Redis, `kick:<instanceId>`, TTL 30с).

Сериализует bookkeeping одного инстанса. Захват — `redis.withLock` (уходит, если занят) для `kickOnce`; `withInstanceLock` (ПЕТРАИТ ~1.4с) для критичных советов, где сигнал нельзя потерять (коммит ю, хуки, таймеры, решения). `Owner-token` (Lua `compare-and-del`) — лок нельзя «украсть».

2. АРЕНДА ШАГА (`leasedUntil` в БД, 200с).

Держит ю-шаг занятым на время внешнего вызова, ДАЖЕ если инстанс-лок протух.

Ставится со `status-guard` на свободную аренду (нет двойной аренды).

3. STATUS-GUARD'Ы В `updateMany` (Postgres).

Каждый переход — `updateMany(WHERE status=ожидаемый). count=0` = «проиграл гонку, ничего не делаем». Нет UPDATE-без-WHERE-гварда во всех переходах статусов (инстанс, шаг, `claim`, эскалация, `join-депозит`).

Правило первого замка: `advance/cancel/fail` ВСЕГДА берут инстанс ПЕРВЫМ (`updateMany instance`) — единый порядок → нет дедлоков и осиротевших шагов.

9.13 ОТМЕНА И ОШИБКА ИНСТАНСА

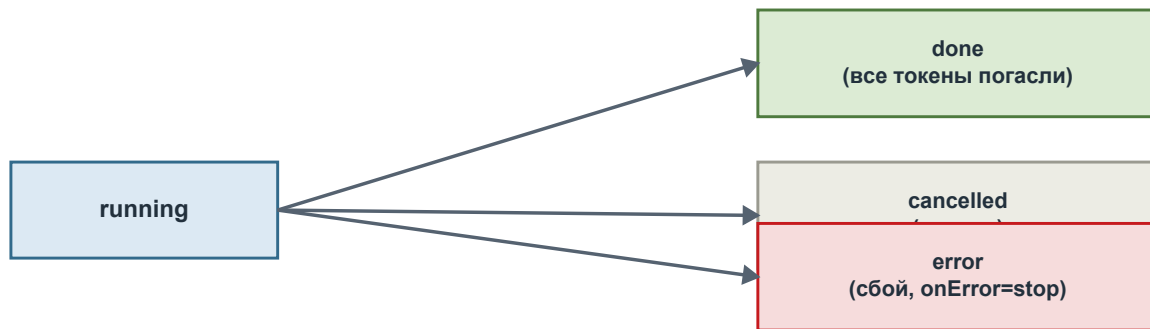
cancelInstance(instanceId, byUserId): в транзакции – захватить инстанс

(`updateMany running→cancelled`), ВНУТРИ транзакции выбрать активные шаги с задачами, отменить все активные шаги. ПОСЛЕ — отменить открытые задачи от имени инициатора, эмитнуть `process.cancelled`. (Шаги выбираем внутри tx после захвата — снимок «до» мог бы отменить уже принятую задачу.)

failInstance(instanceId, stepId, message): `markErrorTx (running→error, шаг→error, прочие active→cancelled)` + `afterFailed`.

markErrorTx(tx, ...): перевод в `error` ВНУТРИ уже открытой транзакции (без `after-эффектов`) — используется и напрямую из `completeStepAndAdvance` при «выход никуда не ведёт» / «лимит шагов».

9.14 СТАТУСЫ И ПЕРЕХОДЫ



терминальные статусы необратимы

Инстанс: running → done | cancelled | error. Терминальные — необратимы.

running	все токены погасли (drain) cancelInstance failInstance / markErrorTx	done cancelled error
---------	--	---

Шаг: active → done | error | cancelled.

active → **done** (completeStepAndAdvance / хук / claim / решение / таймер)

active → **cancelled** (отмена инстанса / markErrorTx гасит соседей)

active → **error** (сам шаг упал с onError='stop')

10. ПОТОК ДАННЫХ (variables, items[], setVariables, {{...}})

Данные процесса живут в `ProcessInstance.variables` (JSON). Туда попадают:

- Анкета запуска (поля form).
- Данные триггера (тело вебхука, поля Telegram-сообщения, payload события).
- Служебные ключи: `_item` (текущий элемент цикла), `_loopIdx_<nodeId>` (индекс цикла), `_subprocessDepth` (глубина под-процессов), поля из ноды «Задать данные».

Как нода ПИШЕТ в данные: возвращает `{kind:'complete', setVariables:{ключ:значение}}`. Движок в advance-транзакции мержит их в variables (read-merge-write). Это делают «Задать данные» (`data.set`) и «Перебрать список» (`loop.each`).

Как нода ЧИТАЕТ данные: две подстановочные функции в ctx:

- **ctx.render(text)** — подставляет `{{...}}` в СТРОКУ (для title/URL/текста).
- **ctx.resolveValue(expr)** — возвращает СЫРОЕ значение (массив/число/объект) — для источника списка цикла и правых частей «Задать данные».

Контекст подстановок (`renderCtx`), доступные корни:

form.<поле> — анкета/переменные инстанса.

steps.<nodeId>... — output завершенного шага (грузится, только если конфиг ноды реально ссылается на steps — оптимизация P1).

item.<поле> — текущий элемент цикла (из `variables._item`).

initiator.name — имя инициатора.

instance.name — имя процесса.

Про «items[]»: в отличие от `n8n`, где по проводу течёт МАССИВ элементов, у нас по проводу течёт ОДИН токен, а «список» — это обычные данные (массив в `variables/output` прошлого шага), который перебирает явная нода `loop.each`. Причина — наличие `human/wait`-шагов, где авто-fan-out на элемент был бы опасен. Это осознанный компромисс.

Пример потока:

```

[HTTP: заказы Kaspi] output.body.data = [ {id,sum}, {id,sum}, ... ]
      | success
      ▼
[Перебрать список] source = {{steps.kaspi.body.data}} → _item = {id,sum}
      | loop
  
```

```

▼
[Задать данные]  итог = item.sum * 1.12  → variables.итог
|
▼
[Уведомить]  «Заказ {{item.id}} на {{form.итог}} Т»
| (провод обратно в «Перебрать список»)
▼
... пока элементы не кончатся → done

```

11. ЯЗЫК ВЫРАЖЕНИЙ (грамматика, операторы, функции)

[ФАЙЛ] process-expression.ts · функция evalExpression(src, ctx)

Это НЕ JavaScript и НЕ eval. Тонкий рекурсивный парсер строит AST из замыканий и вычисляет по белому списку. Доступ к данным — ТОЛЬКО own-property (прототип недостижим: {{form.constructor}} вернёт undefined). Ошибка парса/вычисления → исключение, renderTemplate его ловит и подставляет ". Эталон — Google CEL / формулы Salesforce: мощь выражений без поверхности выполнения произвольного кода.

Грамматика (по приоритету, сверху вниз = слабее связывает):

ternary cond ? a : b

or a || b (возвращает первый truthy)

and a && b

equality a == b, a != b (== мягкое: a===b ИЛИ String(a)===String(b))

comparison a < b, >, <=, >= (числовое сравнение)

additive a + b, a - b (+ склеивает строки, если хоть один операнд строка)

multiplicative a * b, /, %

unary !x, -x

postfix a.b a[expr] (доступ к полю/индексу, own-property)

primary число, 'строка', true/false/null, (...), имя, вызов функции

Функции белого списка (FUNCS):

len(x) длина строки/массива (иначе 0)

upper(s)/lower(s)/trim(s) строковые

round(n, d) округление до d знаков

number(x) к числу

string(x) к строке (объект → JSON)

contains(s,sub) подстрока

default(a,b) a, если не пусто/не null; иначе b

Примеры:

item.sum * 1.12

round(item.sum * 1.12, 2)

upper(item.name)

form.budget > 100000 ? 'дорого' : 'ок'

default(item.city, 'Алматы')

contains(lower(form.text), 'срочно')

len(steps.kaspi.body.data)

Где используется:

- Правые части ноды «Задать данные» (data.set): «итог = item.sum * 1.12».
- Источник списка ноды «Перебрать список» (resolveValue).
- Любое {{ ... }} в тексте: renderTemplate вызывает resolveExpr. Плоский путь (form.budget) идёт быстрым path-lookup (ноль риска для старых шаблонов), всё сложнее — через evalExpression.

12. ОБРАБОТКА ОШИБОК (onError, повторы, порт «Ошибка»)

Две универсальные настройки авто-рендерятся в панели любой НЕ триггер / НЕ под-нода /

НЕ терминальной ноды (реестр добавляет поля; компилятор извлекает из raw config):

onError — что делать, если шаг УПАЛ (throw):

'stop' (умолчание) — валить инстанс в error.

'continue' — продолжить по main/success, будто ок (output.continued=true).

'errorOutput' — уйти в порт «error», если он есть у ноды (иначе как continue).

Повторы при сбое (Retry On Fail) – ТОЛЬКО у io-нод (внешний вызов):

retryMaxTries (0–5), retryWaitMs (0–10000). Повторяет и throw, и «обработанный» сбой (нода сама вернула порт 'error'). Идёт ВНЕ инстанс-лока (пауза не держит лок). Число реальных повторов пишется в output._retries (наблюдаемость).

Логика в runNodeWithPolicy (движок):

```
for attempt in 1..(1+retryMaxTries):
  result = run()
  если io и result.outputKey=='error' и есть ещё попытки → пауза, повтор
  иначе → вернуть result (+_retries)
  если throw и есть ещё попытки → пауза, повтор
  всё исчерпано на throw → applyOnError(node, message):
    stop → fail; errorOutput → порт 'error'; continue → main/success
```

Важно: дешёвые ноды (executeStepLocked) вызываются с allowRetry=false — повтор с паузой под инстанс-локом недопустим. Повторы работают только вне лока (io-ноды).

13. ТРИГГЕРЫ (типы, реестр резолверов, условия, анти-runaway)

[ФАЙЛ] process-triggers.service.ts · класс ProcessTriggerRouter

Фиксированного «Старт» нет. Процесс начинается с ТРИГГЕР-НОДЫ. Авто-триггеры при

публикации зеркалятся в таблицу ProcessTrigger (syncTriggersOnPublish). Поуптер:

onModuleInit: подписывается на все PROCESS_EVENT_TYPES в EventBus.

onEvent(type, payload): анти-runaway → резолв воркспейса → фильтр триггеров по индексу (workspaceId, type, enabled, eventType) → entry-condition → startInstance.

runDueSchedules(): из крона; берёт due-расписания ОПУБЛИКОВАННЫХ процессов; сначала сдвигает nextRunAt (claim от двойного запуска), потом стартует.

fireWebhook(token, body): публичный вебхук; тело → анкета.

fireTelegram(token, update): апдейт Telegram → анкета (text/chatId/fromName/...).

Реестр резолверов воркспейса (Ф4) – у части событий нет workspaceId в payload, резолвим по сущности. Новое семейство событий = +1 запись, ядро не трогаем:

'workspace.' → payload.workspaceId

'task.' → task.findUnique(taskId).workspaceId

'shop.order.' → order → shop; воркспейс только если shop.ownerType='workspace' (магазин компании), иначе null (личный магазин не триггерит).

Entry-condition (Ф2, sflow#1): у event-триггера можно задать condField/condOp/ condValue — фильтр по полю события ДО старта (evalCondition). Пусто → запускать всегда.

Анти-runaway (A4) — три защиты от петли «нода→событие→процесс→нода→...»:

1. payload.source==='process' → игнор (событие, порождённое самим движком).

2. Для task.* дополнительно: если taskId принадлежит шагу процесса

(processStepRun по taskId) → это self-событие, игнор.

3. Потолок одновременно бегущих инстансов на воркспейс (maxRunningInstancesPerWorkspace = 1000): авто-запуск сверх лимита тихо пропускается (в startInstanceProgrammatic).

Список событий (PROCESS_EVENT_TYPES): принят сотрудник, уволен, назначена должность, аттестован, задача завершена/создана, оформлен/собран/подтверждён заказ.

14. БЕЗОПАСНОСТЬ (гейты ролей, run-as-initiator, SSRF, сейф кредов)

ГЕЙТЫ РОЛЕЙ (лестница как в Staff). В `processes.service.ts`:

Читать/запускать процессы — команда (trainee+; Подрядчик изолирован).

Редактировать/публиковать/архивировать/валидировать/журнал — manager+.

visibility='admins' — виден и доступен только admin+ (стена enforced на edit/publish/validate/archive/журнал).

Инстансы: manager+ видит все; рядовой — свои запуски + где он исполнитель.

RUN-AS-INITIATOR (главный принцип безопасности действий). BCE ноды-действия (action.richcard, service.message, staff.assign, workspaces.role, process.start) и

создание задач/уведомлений идут ОТ ИМЕНИ ИНИЦИАТОРА (ctx.startedById) через сервисные API. Сервис перепроверяет права ИМЕННО этого пользователя → процесс не может сделать

больше, чем его инициатор (нет привилегий-эскалации). Именно поэтому `staff.assign`

требует, чтобы инициатор был manager+ (assertStaffManage внутри StaffService), а `workspaces.role` — admin+

(assertCanManage). Чтения ИИ-агент делает через core/access (та же видимость, что у пользователя). НЕ лезть в чужие таблицы через `ctx.deps.db` —

это обходит права и денежные инварианты (Принцип 4 платформы).

Отложено сознательно: нода выплаты из кошелька — у payEmployee owner-check только в контроллере, поднять его в сервис прежде чем делать ноду (риск эскалации).

RUNTIME-ПРОВЕРКА ЧЛЕНСТВА. Публикация валидирует состав, но человек мог быть уволен

ПОСЛЕ. assertActiveMember (в builtin-нодах) при исполнении проверяет, что исполнитель/

согласующий/получатель — действующий член (userRole active, не contractor). Уволенный не получит задачу/уведомление.

SSRF-ЗАЩИТА (A12) — самый тонкий момент. HTTP-нода, агентский `http_get`, KZ-коннекторы, 1C — все ходят наружу через `safeFetch`. Релекс по хосту обходится (DNS-rebinding, редирект на metadata, octal/hex/decimal-IPv4, IPv6-mapped), поэтому:

1. Числовой разбор IP во BCEX формах (inet_aton: dec/hex/oct, 1–4 части) + классификация частных диапазонов (loopback, 10/8, 172.16/12, 192.168/16, 169.254 metadata, CGNAT, multicast, IPv6 ULA/link-local/mapped).
 2. DNS-резолв имени и проверка КАЖДОГО адреса ДО соединения (ловит имя→приватный IP).
 3. Ручной follow редиректов с реперепроверкой каждого хопа + снятие Authorization/Cookie при кросс-хост редиректе (как браузеры).
- Разрешены только `http/https`; `localhost/*`. `internal/*`. `local/*`. `localhost` запрещены.
(Полный пин соединения к IP — вне scope; окно rebind узкое.)

СЕЙФ КРЕДОВ (Ф3). `ProcessCredential` хранит секрет в AES-256-GCM (`process-crypto.ts`, ключ из `JWT_SECRET`). `loadCredentialSecret` проверяет workspaceId (кред своей орг) и

расшифровывает. Секрет наружу (в DTO) не отдаётся. Типы: `bearer (token)`, `basic (username/password)`, `header (headerName/headerValue)`. Управление кредитами — manager+.

15. СПРАВОЧНИК НОД — КАЖДАЯ НОДА С ПОЛЯМИ И ПРИМЕРОМ

Формат: ТИП · Название · [категория, иконка] · флаги (auto/io/trigger/join/subNode). Порты: входы → выходы. Поля config. Поведение run(). Пример.

Всего типов нод: 26. Файлы: builtin (15), service (1), action (5), ai (5),

kz (5) — сумма пересекается: telegram считается в kz.

15.1 ТРИГГЕРЫ (категория trigger) — точки входа, все auto, все trigger

start · «Запуск вручную» · [trigger, auto]

Входы: нет. Выходы: main.

Поля: (config пустой; анкета запуска редактируется на этой ноды в редакторе).

run(): просто complete — токен уходит по main. Точка входа по умолчанию.

Пример: базовый ручной процесс «Заявка на отпуск».

trigger.schedule · «По расписанию» · [trigger, auto]

Поля: everyValue (число), everyUnit (hours|days), runAsUserId (от имени).

run(): complete. Реально запускается кронем runDueSchedules по nextRunAt.

Пример: каждые 6 часов опрашивать новые заказы Kaspi.

trigger.webhook · «Веб-хук» · [trigger, auto]

Поля: runAsUserId. После публикации в панели появляется публичный URL.

Запуск: POST на /api/processes/webhook/:token; тело → анкета.

Пример: сайт отправляет заявку → процесс её обрабатывает.

trigger.event · «Событие в SuperApp» · [trigger, auto]

Поля: eventType (из PROCESS_EVENT_TYPES), condField/condOp/condValue (условие, необяз.), runAsUserId.

Запуск: EventBus → onEvent; payload → анкета; entry-condition фильтрует.

Пример: «принят новый сотрудник» → запустить онбординг.

trigger.telegram · «Telegram: входящее» · [trigger, auto]

Поля: credentialId (bearer-токен бота @BotFather), runAsUserId.

Запуск: сообщение боту → POST /api/processes/webhook/telegram/:token →

{{form.text}}, {{form.chatId}}, {{form.fromName}}, {{form.fromId}},

{{form.messageId}}. При публикации бот-вебхук ставится автоматически (best-effort).

Пример: бот-ассистент: входящее → AI-Агент → ответ нодой «Telegram».

15.2 ЛЮДИ (категория people)

human.task · «Задача человеку» · [wait: auto=false]

Выходы: main.

Поля: title, description, assigneeMode (member|department|initiator), assigneeUserId (если member), departmentId (если department), dueInHours (SLA).

run():

– department → очередь: НЕ создаёт задачу; уведомляет членов отдела; wait с patch.departmentId; output — спецификация будущей задачи (claim читает отсюда).

– member/initiator → создаёт РЕАЛЬНУЮ задачу Задачника (createTask от имени инициатора, skipEnvironmentChecks+origin='process'); wait с patch.taskId.

Проверяет assertActiveMember.

Приёмка задачи → onTaskCompleted → токен дальше.

Пример: «Найти стиральную машину до {{form.budget}} Т» на отдел снабжения.

human.approval · «Одобрение» · [wait: auto=false]

Выходы: approved, rejected.

Поля: title, assigneeMode (member|initiator), assigneeUserId, dueInHours.

run(): проверяет assertActiveMember(согласующий); уведомляет его; wait с assigneeId. decideApproval ведёт токен по approved/rejected. Отклонение можно вернуть назад петлём-связью.

Пример: одобрить покупку дороже лимита у руководителя.

staff.assign · «Назначить должность» · [io, auto]

Выходы: success, error.

Поля: userId, positionId, branchId (необяз.).

run(): getService('StaffService').assignPosition(инициатор, ws, userId, {...}).

Инициатор должен быть manager+ (проверяет StaffService).

Пример: при найме автоматически выдать должность «Стажёр-кассир».

workspaces.role · «Сменить роль сотрудника» · [io, auto]

Выходы: success, error.

Поля: userId, role (trainee|staff|manager|admin).

run(): getService('WorkspacesService').updateMember(инициатор, ws, userId, {role}).

Инициатор должен быть admin+ (владельца/подрядчика назначить нельзя).

Пример: после аттестации повысить Стажёра до Сотрудника.

15.3 ЛОГИКА (категория flow)

condition · «Если» · [auto]

Выходы: true («Да»), false («Нет»).

Поля: field (поле анкеты), op (=/>/</>=/</>contains/empty/not_empty), value (кроме empty/not_empty).

run(): evalCondition(variables[field], op, value) → outputKey 'true'|'false'.

validateConfig: проверяет, что field есть в анкете.

Пример: если budget > 100000 → ветка «одобрение», иначе «купить сразу».

parallel.split · «Развилка» · [auto, multiOut]

Выходы: main (ведёт к ≥2 нодам). run(): complete. Порождает токен на каждое ребро.

Пример: параллельно «оформить документы» и «выдать технику».

parallel.join · «Слияние» · [wait: auto=false, join]

Выходы: main. run(): arrivals≥expected → complete, иначе wait.

Пример: дождаться и документов, и техники, затем «завершить онбординг».

delay · «Пауза» · [wait: auto=false]

Выходы: main. Поля: amount, unit (minutes|hours|days).

run(): wait с deadlineAt=now+Δ. Добивает крон runDueTimersAndEscalations.

Пример: подождать 3 дня после найма, затем напомнить о документах.

loop.each · «Перебрать список» · [auto]

Выходы: loop («Каждый»), done («Готово»).

Поля: source (ссылка на массив, напр. {{steps.kaspi.body.data}}), maxIterations.

run(): см. 9.7. Ветку «Каждый» замкнуть проводом обратно в эту ноду.

Пример: перебрать все новые заказы Kaspi и по каждому создать задачу.

data.set · «Задать данные» · [auto]

Выходы: main. Поля: assignments (многострочно, «имя = выражение»).

run(): для каждой строки «ключ = выражение» вычисляет resolveValue(выражение) и кладёт в setVariables → мержится в variables. Дальше — {{form.ключ}}.

Пример: «итог = item.sum * 1.12», «имя = upper(item.name)».

end · «Конец» · [auto, terminal]

Входы: main. Выходов нет. run(): complete → 'consumed' (гасит свой токен, см. 9.6).

15.4 СЕРВИСЫ (категория service)

notify · «Уведомить» · [auto] + astool (инструмент агента)

Выходы: main, astool (ai_tool).

Поля: to (initiator|member), userId (если member), title, message.

run(): best-effort — любой сбой (нет заголовка/получателя/уволен) НЕ валит процесс, пишет в output.skipped и идёт по main.

Как инструмент агента (tool notify_initiator): агент сам решает уведомить инициатора.

Пример: «Онбординг {{initiator.name}} завершён».

service.message · «Сообщение в чат» · [io, auto] + astool

Выходы: success, error, astool.

Поля: to (member|chat), userId (личка), chatId (в чат), text.

run(): getService('MessengerService'); member → openDm(инициатор, userId) → chatId; затем sendMessage(инициатор, chatId, text). Инициатор должен иметь доступ к чату.

Как инструмент агента (tool send_message): текст придумывает агент.

Пример: написать сотруднику в личку от имени инициатора.

action.richcard · «Действие с карточкой» · [io, auto]

Выходы: success, error.

Поля: actionKey (order.confirm/task.accept/event.rsvp_*/listing.buy/...), refType (order|listing|crowdfunding|task|event), refId, payload (JSON).

run(): getService('RichCardsService').execute(инициатор, actionKey, {type,id}, payload). Права перепроверяются rich-cards (capability recheck).

Пример: авто-подтвердить заказ Kaspi после проверки оплаты.

process.start · «Запустить процесс» · □ · [io, auto]

Выходы: success, error.

Поля: definitionId (id опубликованного процесса той же орг), input (анкета JSON).

run(): getService('ProcessesService').startSubprocess(ws, defId, инициатор, input, depth+1). depth из variables._subprocessDepth; защита от рекурсии (maxSubprocessDepth=5).

Пример: из «онбординга» запустить под-процесс «выдача техники».

15.5 ИНТЕГРАЦИИ (категория integration) — все io, все auto, success/error

service.http · «HTTP-запрос» · [io] + astool (http_get)

Поля: method (GET/POST/PUT/PATCH/DELETE), url, headers (JSON), body, credentialId (из сейфа).

run(): SSRF-safe (assertPublicUrl + safeFetch). Креды: bearer→Authorization, basic→Basic, header→свой заголовок. Ответ (JSON или текст, обрез 100КБ) в output.body; outputKey success/error по res.ok. Подстановки в URL/теле/заголовках.

Как инструмент агента (http_get): агент подставляет URL (только GET, только чтение, жёсткий SSRF — LLM prompt-injectable).

Пример: GET заказов из внешней CRM.

kz.telegram · «Telegram» · [io] + astool (send_telegram)

Поля: credentialId (токен @BotFather), chatId ({{form.chatId}}/id/@канал), text.

run(): sendMessage через api.telegram.org (parse_mode HTML).

Как инструмент агента: агент придумывает текст, chatId из ноды.

Пример: ответить пользователю в Telegram (пара с trigger.telegram).

kz.whatsapp · «WhatsApp» · [io]

Поля: credentialId (access-токен), phoneNumberId, to, text.

run(): WhatsApp Cloud API (graph.facebook.com). Текст в 24ч-окне.

kz.sms · «SMS (Mobizon)» · [io]

Поля: credentialId (apiKey Mobizon), recipient, text, from (альфа-имя).

run(): api.mobizon.kz sendSmsMessage; success если res.ok И body.code===0.

kz.kaspi · «Kaspi Магазин» · [io]

Поля: credentialId (X-Auth-Token), operation (new_orders|accept|complete), orderId.

run(): kaspi.kz/shop/api/v2/orders. new_orders — GET NEW+APPROVED_BY_BANK;

accept/complete — POST со статусом. Вебхуков у Kaspi нет → опрос по расписанию.

kz.odata · «1C (OData)» · [io]

Поля: credentialId (basic логин/пароль), baseUrl, entity, operation (list|create), filter (\$filter), body (JSON).

run(): Basic-auth к опубликованной базе 1C; assertPublicUrl (база должна быть публичной). list → GET ?\$format=json&\$filter=...; create → POST.

15.6 AI (категория ai)

ai.generate · «AI» · [io, auto]

Выходы: success, error.

Поля: provider (anthropic|openai|openai-compatible), credentialId (bearer-ключ), model, baseUrl (для совместимых), systemPrompt, userPrompt, temperature, maxTokens.

run(): resolveLlmConfig (ключ из сейфа) → llmGenerateText. Результат в output.text.

Подстановки {{form.x}}/{{steps.x}}.

Пример: «Сократи: {{steps.fetch.body}}».

ai.agent · «AI-Агент» · [io, auto, cluster] + astool

Входы: main, ai_model (Модель), ai_memory (Память), ai_tool (Инструменты, multi), ai_output (Парсер). Выходы: success, error, astool.

Поля: systemPrompt, userPrompt, toolDescription (когда агент = инструмент), maxIterations (1–8).

run(): движок собрал ctx.cluster (buildAgentCluster). runAgentWithCluster: память + tool-calling-цикл + парсер. Нет Модели → error. Результат в output.text, output.toolCalls, output.data (если парсер). Агента можно подключить инструментом к другому агенту (оркестратор→специалисты, глубина ≤3).

Пример: диспетчер снабжения, у которого инструменты HTTP + Telegram + Уведомить.

ai.model · «Модель» · [subNode]

Выход: model (ai_model). Поля: как llmFields. run(): поор (данные читает движок).

Одну Модель можно подключить к нескольким агентам.

ai.memory · «Память» · [subNode]

Выход: memory (ai_memory). Поля: sessionKey (пусто = id запуска), window

(сколько реплик помнить). `run()`: поор. Диалог хранится в Redis LIST по ключу сессии (помнит между запусками с тем же ключом).

ai.parser · «Структурированный ответ» · [subNode]

Выход: `parser (ai_output)`. Поля: `fields` («ключ: описание» по строке).

`run()`: поор. Заставляет агента вернуть строгий JSON; результат в `output.data` (доступен как `{{steps.агент.data.поле}}`).

16. СПРАВОЧНИК ФУНКЦИЙ — ПО ФАЙЛАМ, ЧТО / КАК / ЗАЧЕМ

Здесь — каждая значимая функция: сигнатура (упрощённо), что делает, как, зачем.

16.1 process-engine.service.ts (ДВИЖОК)

`startInstance(params) : Promise<string>`

ЧТО: создаёт инстанс и стартовый токен, эмитит `process.started`, зовёт `kick`.

КАК: кэширует план; определяет `entryNodeId` (сработавший триггер или `startNodeId`); в транзакции `create(instance running) + create(step active на entry-ноде)`.

ЗАЧЕМ: единая точка рождения инстанса (и ручной запуск, и все триггеры).

`kick(instanceId) : Promise<void>`

ЧТО: крутит `kickOnce` до `maxAutoChain` раз или до «стоп».

ЗАЧЕМ: за один толчок «разобрать» все готовые ветки; бюджет — от заикливания.

`kickOnce(instanceId) : Promise<boolean> *`

ЧТО: один проход движка (3 фазы — см. 9.3).

КАК: Фаза1 под локом (выбор+аренда/дешёвое исполнение); Фаза2 io вне лока; Фаза3 коммит под локом. Возврат `true`=дальше / `false`=стоп.

ЗАЧЕМ: не держать лок поверх сети (P3), но сериализовать `bookkeeping`.

`executeStepLocked(instance, plan, step)`

ЧТО: исполнить+закоммитить ДЕШЁВУЮ ноду под уже удерживаемым локом.

КАК: `runNodeWithPolicy(allowRetry=false) → commitResult`; ошибка → `failInstance`.

ЗАЧЕМ: `bookkeeping`-ноды без сети быстрее прогнать под локом, чем арендовать.

`runIoStep(instance, plan, step)`

ЧТО: исполнить io-ноду вне лока (Фаза2) и закоммитить под локом (Фаза3).

КАК: `runNodeWithPolicy(allowRetry=true)` БЕЗ лока; затем `withInstanceLock → commitResult`. Не взял лок → предупреждение (крон переисполнит по истечении аренды).

ЗАЧЕМ: долгий внешний вызов не держит инстанс-лок, но результат приземляется.

`runProvider(instance, plan, step, node) : Promise<NodeRunResult>`

ЧТО: собрать `ctx` (`buildContext + join + cluster`) и вызвать `provider.run()`.

ЗАЧЕМ: единое место сборки контекста для любой ноды.

`runNodeWithPolicy(instance, plan, step, node, allowRetry)`

ЧТО: выполнить ноду с повторами и `onError` (см. глава 12).

ЗАЧЕМ: универсальная надёжность io-нод; аккуратный `fallback` при `throw`.

`withRetryMeta(result, retries)` – помечает `output._retries` (наблюдаемость).

`applyOnError(node, message)` – маршрутизация упавшей ноды по `onError` (см. 12).

`sleep(ms)` – пауза (клампится 0..10000мс).

`commitResult(instance, plan, step, result)`

ЧТО: применить результат ноды ПОД удерживаемым локом.

КАК: `wait → commitWait`; `complete → completeStepAndAdvance`.

commitWait(instance, step, result)

ЧТО: «усыпить» шаг (activated=true, аренда снята, patch полей).

КАК: updateMany со status-guard; count=0 при taskId → гасит задачу-сироту.

ЗАЧЕМ: ожидание переживает рестарт; отмена инстанса не оставляет висячих задач.

completeInstanceIfDrained(instanceId)

ЧТО: если активных шагов не осталось — инстанс done (со status-guard) → afterFinished.

ЗАЧЕМ: реализация branch-local end / drain (9.6).

withInstanceLock(instanceId, fn) : Promise<boolean>

ЧТО: выполнить критическую секцию под инстанс-локом с ПЕТРАЕМ захвата (~1.4с).

ЗАЧЕМ: внешний сигнал (приёмка/решение/таймер/коммит io) обязан приземлиться (A7), а не уйти, как бесхитростный kickOnce.

completeStepAndAdvance(instanceId, plan, stepId, nodeId, outputKey, output?, extra?, setVariables?) : Promise<AdvanceOutcome> **

ЧТО: завершить шаг и продвинуть токен — атомарно, со статус-гвардами (см. 9.4).

ЗАЧЕМ: сердце движения токена; исключает двойное продвижение и осиротевшие шаги.

onTaskCompleted / onTaskDeleted / onTaskCancelled(taskId)

ЧТО: хуки Задачника (см. 9.9). Completed → advance+kick; Deleted/Cancelled → fail.

claimQueueStep(userId, instanceId, stepId) : Promise<string>

ЧТО: забрать задачу отдела из очереди, создать реальную задачу (см. 9.10).

decideApproval(userId, instanceId, stepId, decision)

ЧТО: вынести решение по одобрению; токен по ветке approved/rejected (см. 9.10).

reassignStep(instanceId, stepId, newUserId)

ЧТО: переназначить исполнителя активной задачи-шага (manager+).

runDueTimersAndEscalations()

ЧТО: добить истёкшие паузы + эскалировать просроченные человеческие шаги (см. 9.11).

sweepStuckJoins()

ЧТО: крон-добивка зависших слияний по reachability (см. 9.5).

tryForceFireStuckJoin(instanceId, plan, activeSteps) : Promise<boolean>

ЧТО: синхронная (под локом) добивка зависшего слияния в kickOnce. true=сработало.

canReach(plan, from, target) : boolean

ЧТО: BFS по adjacency — может ли живой токен из from дойти до target (слияния).

cancelInstance(instanceId, byUserId) / failInstance(instanceId, stepId, message) /

markErrorTx(tx, instanceId, stepId, message)

ЧТО: терминальные переходы инстанса (см. 9.13).

afterFinished(instanceId) / afterFailed(instanceId)

ЧТО: after-эффекты вне транзакций — уведомления + события шины. afterFailed шлёт и инициатору, и СОЗДАТЕЛЮ определения (sfflow#4 — для авто-запусков).

getPlan(versionId) : Promise<CompiledPlan|null>

ЧТО: план из LRU-кэша или из ProcessVersion.compiled. LRU: delete+set двигает ключ в конец Map (недавно использован).

cachePlan(versionId, plan)

ЧТО: положить план в LRU с потолком 100 (вытесняет наименее недавний = первый ключ).

ЗАЧЕМ: версии копятя вечно (publish создаёт новую каждый раз) — без потолка утечка.

isAutoType(nodeType) : boolean — auto-флаг типа из реестра.

buildAgentCluster(instance, plan, agentNodeId, depth) : Promise<AgentCluster>

ЧТО: собрать оснастку агента — Модель (из сейфа), Память (Redis LIST по ключу сессии), Инструменты (под-ноды-инструменты + под-агенты рекурсивно), Парсер.

КАК: `depth>3` → ошибка; Память через `client.rpush/lrange/ltrim/expire` (НЕ `read-modify-write` JSON — параллельные `append` не теряют реплики, P12).
ЗАЧЕМ: `n8n` cluster-модель: агент = мозг, оснастка приходит портами.

`getInstanceMeta(instance) : Promise<{initiatorName, definitionName}>`
ЧТО: стабильные имена инициатора/процесса, кэш на инстанс (потолок 200).
ЗАЧЕМ: P1 — `buildContext` не делает 2 запроса на КАЖДУЮ ноду.

`buildContext(instance, step, node) : Promise<NodeRunContext>`
ЧТО: собрать контекст ноды: `variables`, `steps` (грузятся ТОЛЬКО если конфиг ссылается на `{{steps}}` — иначе 0 запросов), `render/resolveValue`, `deps`.
ЗАЧЕМ: всё, что нужно ноде для `run()`; оптимизирован под «нет ссылок → нет запросов».

`resolvePath(ctx, path)` — плоский путь `a.b.c` по `own-property` (быстрый путь).
`resolveExpr(ctx, raw)` — снять `{{ }}` → плоский путь (`fast`) ИЛИ `evalExpression`.
`renderTemplate(text, ctx)` — заменить все `{{...}}` на строки (объект→JSON, `bool`→да/нет).

16.2 process-compiler.ts

`compileProcessDocument(doc, registry)` — документ → `{plan, issues}` (см. глава 8).
`extractErrorHandling(config)` — вытащить `onError/retryMaxTries/retryWaitMs` из `raw` `config` с клампингом границ (реестр стрипает их из паспорта, компилятор возвращает).
`inputType/outputType(nodeType, port)` — тип порта ноды (для проверки совместимости).
(внутри) 3-цветный DFS — детект циклов агентов-инструментов (A6).

16.3 process-expression.ts

`tokenize(src)` — строка → токены (`num/str/id/op`); ловит незакрытые строки, чужие символы.
`class Parser` — рекурсивный спуск (`ternary→or→and→equality→comparison→additive→multiplicative→unary→postfix→primary`), строит замыкания-вычислители.
`evalExpression(src, ctx)` — распарсить и вычислить; бросает при ошибке.
`FUNCS` — белый список функций (`len/upper/lower/trim/round/number/string/contains/default`).
`member(obj, key)` — `own-property` доступ (без прототипа).
`num(x)/truthy(x)` — приведения.

16.4 process-node.registry.ts

`register(provider) / get(type) / all()` — реестр типов нод (`Map`).
`listTypes(includeSystem)` — сериализованные паспорта для палитры: стрипает серверные поля (`configSchema/auto/io`); НЕ триггерам/под-нодам/терминалам добавляет поле `onError`, `io-нодам` — `retryMaxTries/retryWaitMs` (авто-рендер в NDV).

16.5 process-triggers.service.ts

`onModuleInit()` — подписка на все `PROCESS_EVENT_TYPES`.
`workspaceResolvers[] / resolveWorkspace(type, payload)` — реестр «префикс→воркспейс».

`onEvent(type, payload)` – анти-runaway → воркспейс → триггеры по индексу → условие → старт.

`runDueSchedules()` – due-расписания опубликованных; `claim nextRunAt` → старт.

`fireWebhook(token, body)` – публичный вебхук → анкета → старт.

`fireTelegram(token, update)` – Telegram-апдейт → анкета (`text/chatId/...`) → старт.

16.6 process-builtin-nodes.ts (хелперы)

`assertActiveMember(ctx, userId, who)` – runtime-проверка членства (уволенный не получит задачу/уведомление).

`departmentMemberIds(ctx, departmentId)` – члены отдела (с подотделами) из `relationTuple`.

`deadlineFrom(hours)` – `now + hours` (для SLA/срока).

`evalCondition(raw, op, expected)` – сравнение значения с константой (нода «Если» + `entry-conditions` триггеров). Умное приведение чисел/строк/булей.

16.7 process-service-nodes.ts (SSRF + креды)

`loadCredentialSecret(ctx, credentialId)` – расшифровать кред своей орг.

`credentialKey(secret)` – достать `token/headerValue/password`.

`isPrivateIpv4Num` / `parseLooseIpv4` / `isPrivateIpv6` / `literalHostIsPrivate` — классификация частных IP во всех формах (`inet_aton`, `hex/oct`, `IPv6-mapped`).

`assertPublicUri(raw)` – синхронная проверка (протокол + литеральный IP/имя).

`assertResolvedPublic(hostname)` – DNS-резолв + проверка каждого адреса.

`safeFetch(url, init, opts)` – * единый SSRF-безопасный выход наружу (ручной `follow` редиректов + реперепроверка хопов + снятие `Authorization` кросс-хост).

`fetchJson(url, init, timeout)` – `safeFetch` + парс JSON (обрез 100КБ).

16.8 process-ai-nodes.ts / process-action-nodes.ts / process-kz-nodes.ts (хелперы)

`resolveLlmConfig(ctx, cfg)` – модель + ключ из сейфа (нода AI и под-нода Модель).

`parserInstruction(fieldsText)` – «ключ: описание» → инструкция «верни строгий JSON».

`extractJson(text)` – достать JSON из ответа LLM (снимает `markdown-обёртку`).

`runAgentWithCluster(cluster, userPrompt, maxIter)` – * выполнить агента (память + `tool-calling` + парсер); переиспользуется для под-агентов.

`sendTelegram(ctx, credId, chatId, text)` – `sendMessage` через Telegram Bot API.

`sendMessageImpl(ctx, text)` – общий код ноды «Сообщение в чат» (в потоке и как `tool`).

`done(ok, output)/fail(message)` – унифицированный финал KZ/action-нод.

17. СКВОЗНЫЕ ПРИМЕРЫ (РЕЦЕПТЫ)

РЕЦЕПТ 1 — Опрос заказов Kaspi и задача на каждый (расписание + цикл + действие).

```
[По расписанию: 30 мин] → [Kaspi: получить новые заказы] —success— [Перебрать список]
                        source={{steps.kaspi.body.data}}                | loop
                        └────────────────────────────────────────────────┘
                        |
                        ▼
[Задача человеку: «Собрать заказ {{item.attributes.code}}» на отдел «Склад»]
| (приёмка → токен возвращается в «Перебрать список»)
...
[Перебрать список] —done— [Конец]
```

Что происходит: крон стартует процесс; HTTP тянет новые заказы; цикл идёт по каждому;

РЕЦЕПТ 2 — Онбординг сотрудника (событие + параллель + слияние).



Инстанс-лок TTL 30с: аренда шага 200с: retrv пауза ≤ 10 с: retrv попыток ≤ 5 .

- `loop.each` — ПОСЛЕДОВАТЕЛЬНЫЙ, не параллельный `per-item`.
- По проводу течёт ОДИН токен, не `items[]` (у нас есть `human/wait-шаги`).
- `SSRF`: полный пин соединения к IP не сделан (окно `rebind` узкое).

выплаты из кошелька (поднять owner-check в сервис), триггеры сообщений/календаря (нужен emit + анти-петля), нода «Фильтр» / параллель-на-элемент, allowlist доменов агента, schemaVersion+перекompilация, async webhook-ACK. Плюс отдельный трек «Доски (Kanban)» — упрощённый tier для нетехнических пользователей.

19. FAQ И ЧАСТЫЕ ОШИБКИ

Крон добьёт с forced=true.

В: io-нода выполнялась дважды. О: В штатном режиме — нет (аренда шага). Экстремально редко при протухшем локе на

коммите крон переисполнит по истечении аренды — семантика «as-good-as-it-gets» для не-идемпотентного I/O (как у n8n). Делай внешние вызовы идиempотентными, где можешь.

В: Как передать данные из шага в шаг? О: Читай output прошлого шага через `{{steps.<nodeId>.поле}}` (`nodeId` — id ноды в

документе). Вычисляй новые поля нодой «Задать данные» → потом `{{form.поле}}`.

В: Как добавить НОВУЮ ноду? О: 1) Создай `ProcessNodeProvider {descriptor, run, validateConfig?}`. descriptor:

`type/title/category/icon/tier/outputs/fields/configSchema/auto(/io/tool/inputs)`.

2) Экспортируй в массив нод файла и зарегистрируй в `processes.service (registry)`. Палитра и валидация «загорятся» сами.

3) В `run()` бери сервисы через `ctx.deps.* / ctx.deps.getService('XxxService')`; действия — ОТ ИМЕНИ `ctx.startedById` (сервис перепроверит права). Внешний I/O → `io:true + safeFetch`.

4) Верни `{kind:'complete', outputKey, output, setVariables?}` или `{kind:'wait', patch, output}`.

В: Почему ноды-действия безопасны? О: Все идут `run-as-initiator (ctx.startedById)` через сервисные API, которые перепроверяют права именно инициатора. Процесс не может больше своего инициатора.

В: Где состояние процесса при рестарте сервера? О: В БД (`process_instances + process_step_runs`). Ничего в памяти не теряется; крон

добьёт спящее/застрявшее.

Конец документа. Вопросы по коду — начни с `process-engine.service.ts` (глава 9/16) и `process-compiler.ts` (глава 8). Один абзац на запоминание: «Документ → компилятор → план; движок ползёт токенами (строки БД) по adjacensy плана; дешёвые ноды под локом, io — вне лока под арендой; переходы — атомарные транзакции со status-guard; крон добывает потерянное».
